

ece 260c.

Static Timing Overview

Tom Spyrou 4/2026

Outline

- Use of an STA engine by a designer
 - Why sta
 - Setting up
 - Limitations, false path, delay calc, SI
- Requirements of an STA engine
- Data Structures
- Ensuring the graph is cycle free
- Possible innovation to allow cycles in the graph
- Analysis
 - combinational logic
 - Required time and arrival times
 - Setup analysis (late mode), Hold Analysis (early mode)
 - clocks
 - cycle accounting
- Latches
 - Simple Iteration
 - The epsilon problem and using a latch graph
- Timing exceptions
 - Early approaches and their evolution to the state of the art
- Exception compression
- Incremental Timing

Why Static Timing Analysis

- Dynamic simulation is impossible for even a small chip
 - Assume combination logic only
 - 100 inputs implies 2^{100} vectors needed to verify timing which is about 10^{30} vectors
 - If a simulator could process 10^6 vectors per second this works out to a sim time of 10^{19} days or about 10^{15} years
 - Talk about a verification bottleneck!
- Now add in state elements and the problem of making sure the critical path is actually in the vector set
- STA can analyze such a design in 1 minute
 - There are some issues, but they can be mitigated in practical use
- STA's quality of result is not dependent on the quality of the vector set

What do I pay

- A different kind of setup
- Less accurate delay analysis
- Introduction of false paths due to topological not functional analysis
- Multiple circuit modes take extra effort

Setting up STA requires constraints not vectors

- The clock definitions are the same as clock constraints
- Arrival times can come from the specification or some kind of worst case check over the vectors
- Required Times can come from the specification and the clocks. One can also define off virtual chip clocks to help with this
- Multicycle paths must be defined
- False Paths must be defined
- Analysis cases (like test_enable) must be defined

Most important uses

- Feed it a gate level netlist and .lib timing information
- Report failing paths, build histograms of failing paths, look at worst negative slack and total negative slack
- Time budgeting of top level requirements to blocks
- Create test vectors for critical paths

What doesn't work well

- Hierarchical timing in the final verification loop – there is no way to take SI into account
- SI calculations very conservative
- Worst case timing is done and process variation is modeled very pessimistically
- True delay reporting is slow and can't run during optimization

Requirements of an STA Engine

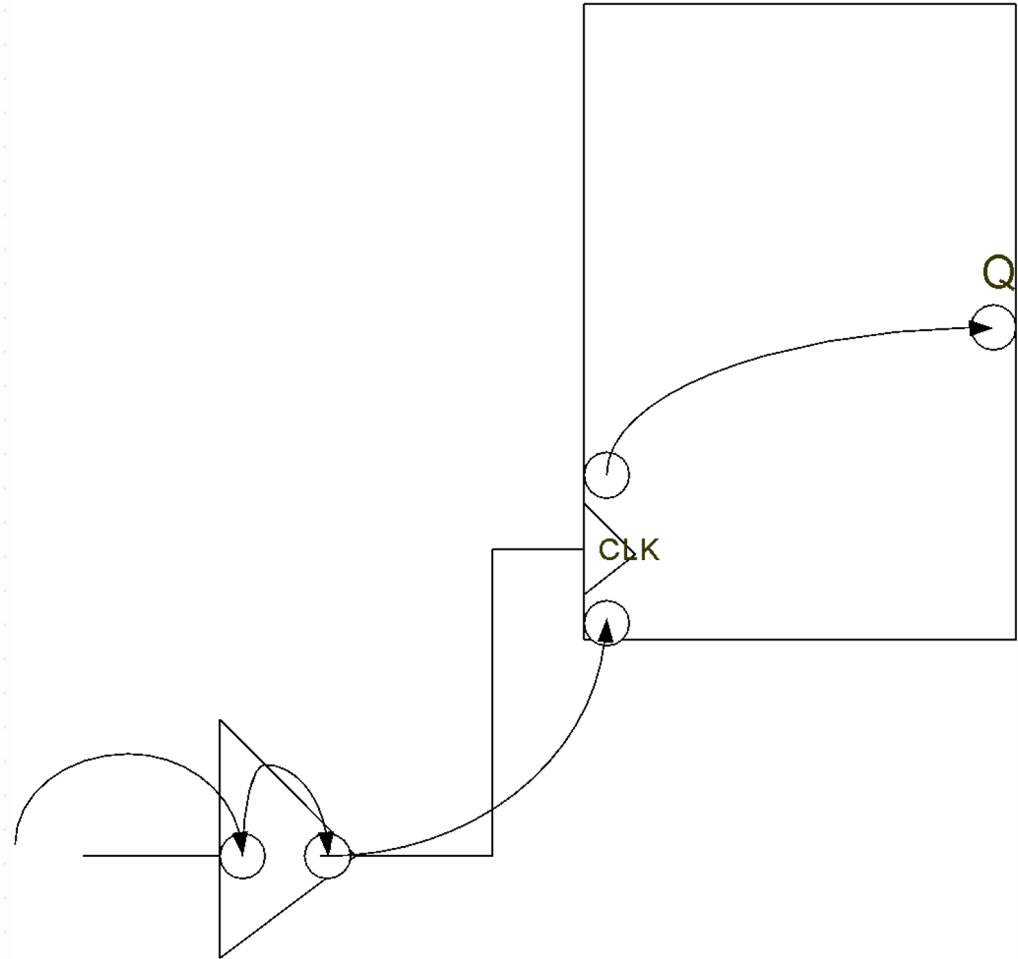
- Must run in linear memory and runtime with circuit size, number of clocks, exceptions, and number of storage elements
- Must support SDC timing constraints
 - Clocks, clock tree assumptions, multi-cycle paths, false paths, path delays, cases and modes
- Must be incremental enough
 - Netlist changes / full retrace on one extreme
 - Query based incremental with limited tracing on the other

Data Structures

- Directed Acyclic Graph
- Gate internal pin to pin timing paths as arcs
- Gate pins as vertices
- Interconnect arcs from gate outputs to gate inputs, block inputs to gate inputs, gate outputs to block outputs
- Described in SDF (standard delay format)
- Some implementations have separate arcs for each transition edge combination for input and output and state dependent timing. Whether an explicit edge or not data must be annotated to the graph somehow
- Split pins on clock inputs to storage elements and block bidirectional pins. Therefore use term timing point to denote vertexes in the graph since they do not always align to pins in the netlist

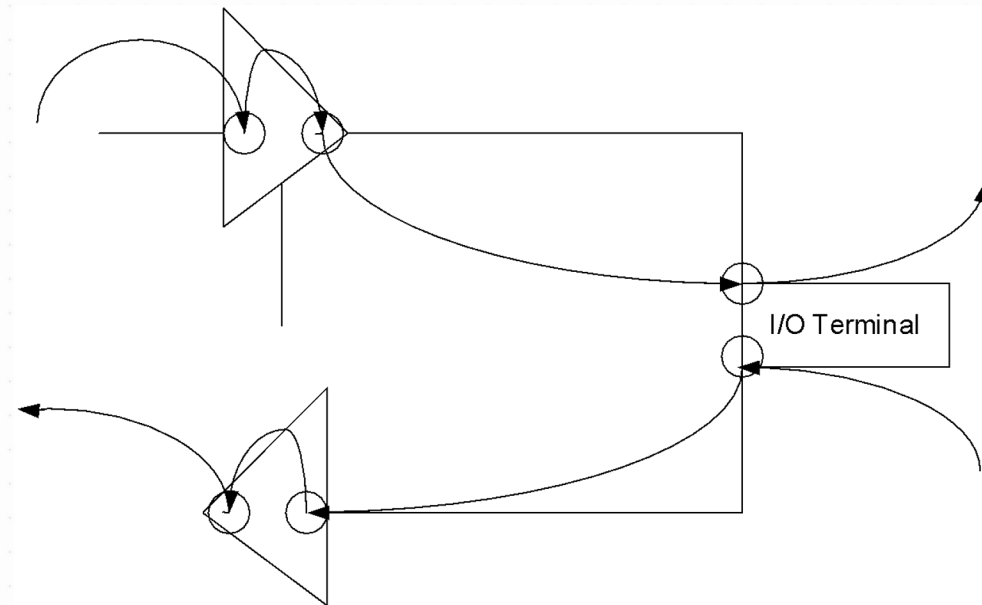
Split Clock Pins Example

- Propagate Clock step terminates at lower timing point on the clock
- Propagate Data starts at upper timing point on the clock
- Allows more simple coding of BFS and storage of clock arrival times for skew analysis



Split Pins Bidirect Example

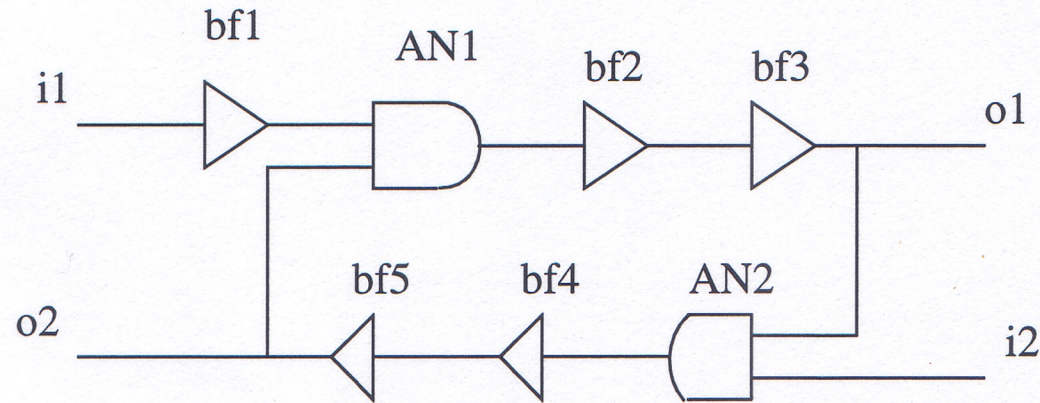
- Threads the graph so bidirectional loop-back does not happen
- If user wants this path, can re-thread
- Note the hierarchical boundary has a split pin
- Timing constraints can go on either side of the split pin



Ensuring the graph is cycle free

- Graph can be scanned in DFS manner in $O(n)$ time to find all loops and automatically break them
- During this process graph can be simultaneously level-ized to prepare for for BFS
- Room should be left between level numbers for addition of gates during optimization
- Breaking loops is bad because it cause the critical path to be missed.

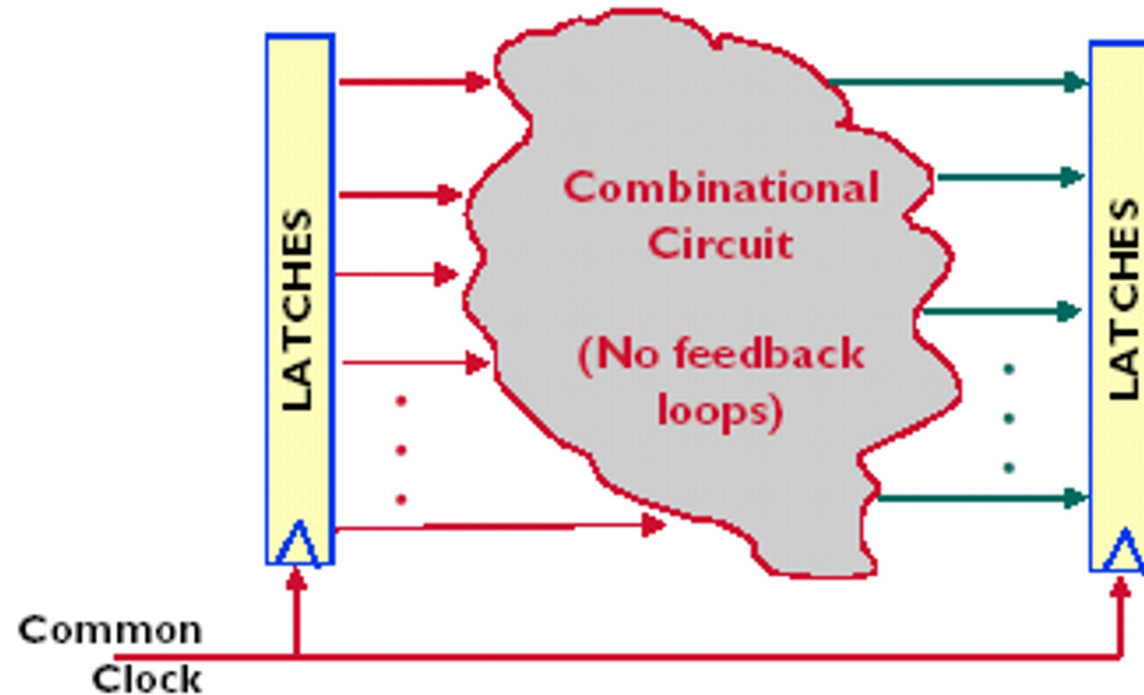
Possible innovation to allow cycles in the graph



- Detect All cycles in the graph
- Use BFS up to cycle entry points
- Use DFS inside the cycle and enumerate paths
- Finish with BFS

Analysis

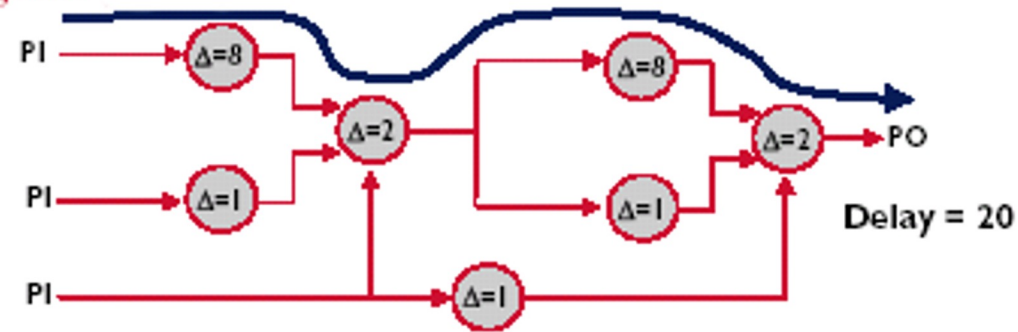
- Every circuits looks the same to STA since it ignores the functions of the logic.



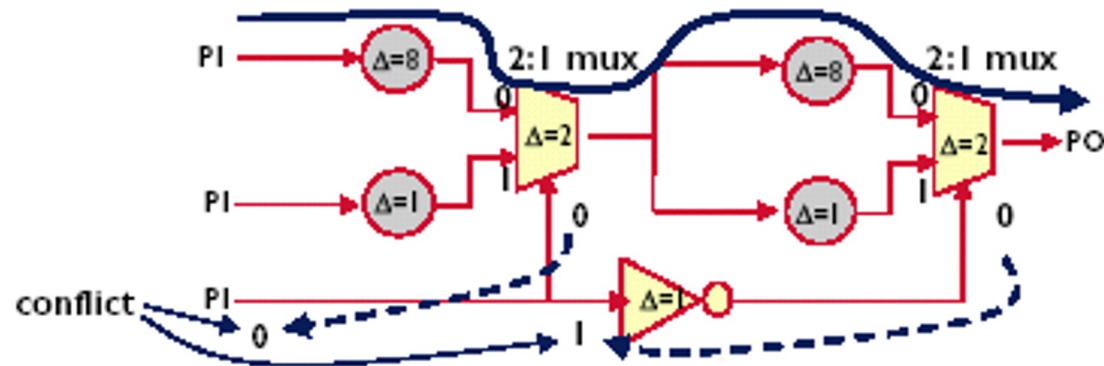
Topological analysis

- Simplifies problem, possibility of reporting false paths

▼ Topological



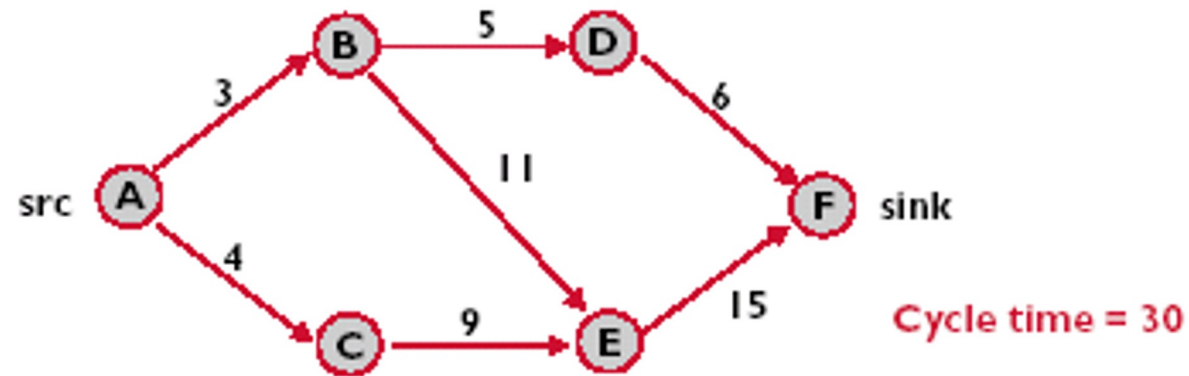
▼ Logical



Required and Arrival Times

- Arrival Time is the time a signal arrives at a pin, found by forward tracing graph adding delays
- Required Time is found by taking the timing constraint at the output pin and tracing backward in the graph subtracting delays
- Setup Analysis is to find paths that are too long and arrive late (late data vs early clock)
- Hold Analysis is to find paths that are too short and arrive early (early data vs late clock)

Example RAT, AT calculation for Early and Late Modes



$$AT_E(E) = 4+9 = 13$$

$$AT_L(E) = 3+11 = 14$$

$$RAT_E(B) = 0-6-5 = -11$$

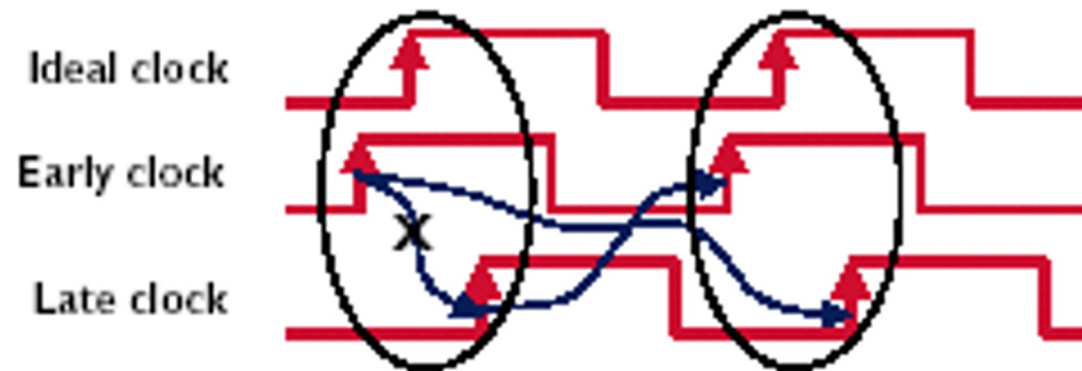
$$RAT_L(B) = 30-11-15 = 4$$

$$Slack_E(B) = 3-(-11) = 14$$

$$Slack_L(B) = 4-3 = 1$$

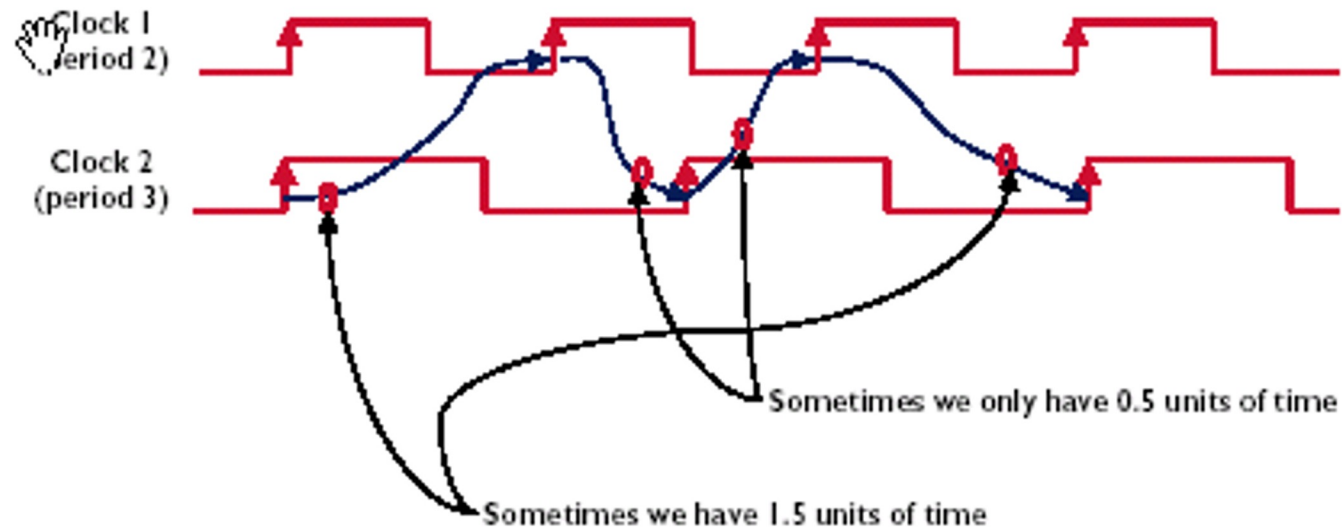
Cycle accounting

- When there is a single clock then this is trivial
 - Required Time on data pin of storage element is clock period minus clock arrival time
 - Delay to Clock is added to arrival time of path from source storage element



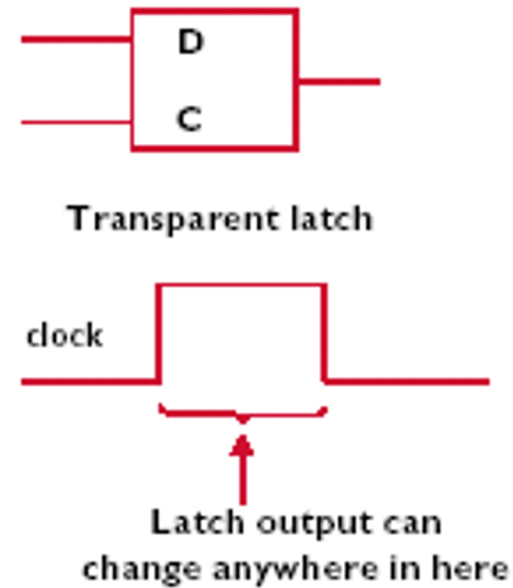
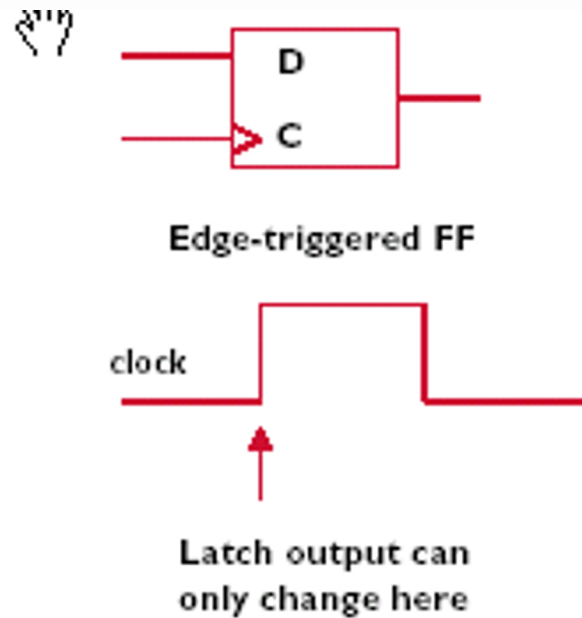
Cycle accounting

- When there are multiple clocks assume data is meant for first possible edge of the ideal clock following the one that launched it.
 - Over-riding this requires use of timing exceptions



Latches

- So far we have not discussed level sensitive latches



Latches

- Launch tracing from all latch clock pins assuming they do not borrow
- If latch needs to borrow schedule its output
- If the latches true setup or hold time is failing clip its arrival time and schedule its output
- Iterate until nothing else gets scheduled
- Failures will be bunch up at last latches
- In optimization this negative slack has to be distributed

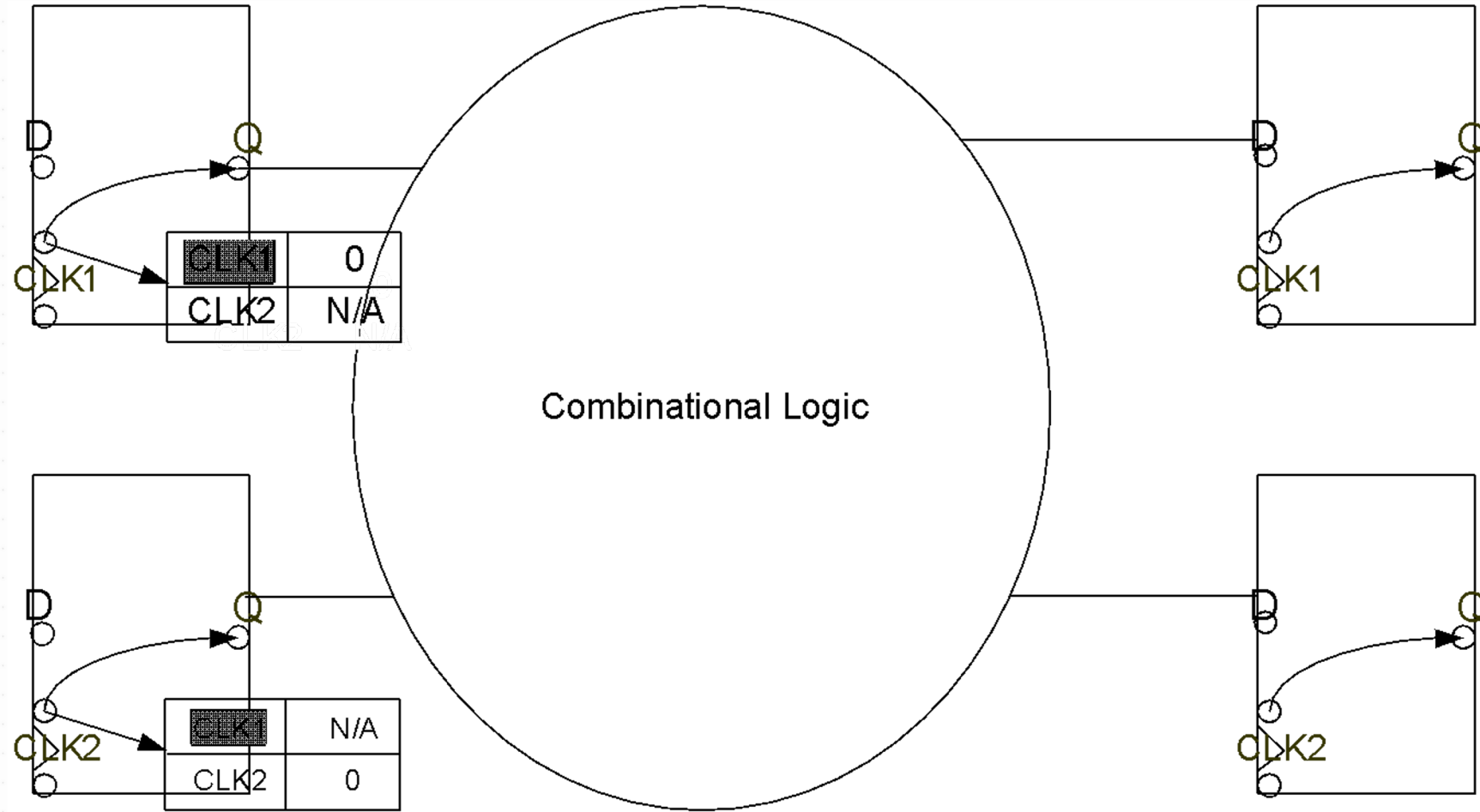
The Basic Search

- The Graph
 - Startpoints are inputs to the circuit and clock inputs to storage elements
 - Endpoints are outputs of circuit and data inputs of storage elements
- Propagate the Clocks
 - For each clock input BFS to all clock data pins
 - Offset startpoint arrival times and end point required times with information from the clock propagation and cycle accounting
- Propagate the Data
 - Use a BFS from startpoints to end points
 - Use multiple timing totals at every pin to take into account multiple clocks and exceptions
 - Can optionally store back pointers to record K critical paths but this time/memory is wasted on optimization programs and should be left to a reporting phase

Multiple Timing Totals

- Simple minded implementation is that each clock and each exception gets its own total
- Memory and runtime increase quickly
 - Occurrence pins are the most common netlist object
 - There can be thousands of exceptions
- At Timing endpoints like totals can be combined and evaluated
- At Timing endpoints point to point exceptions can be evaluated

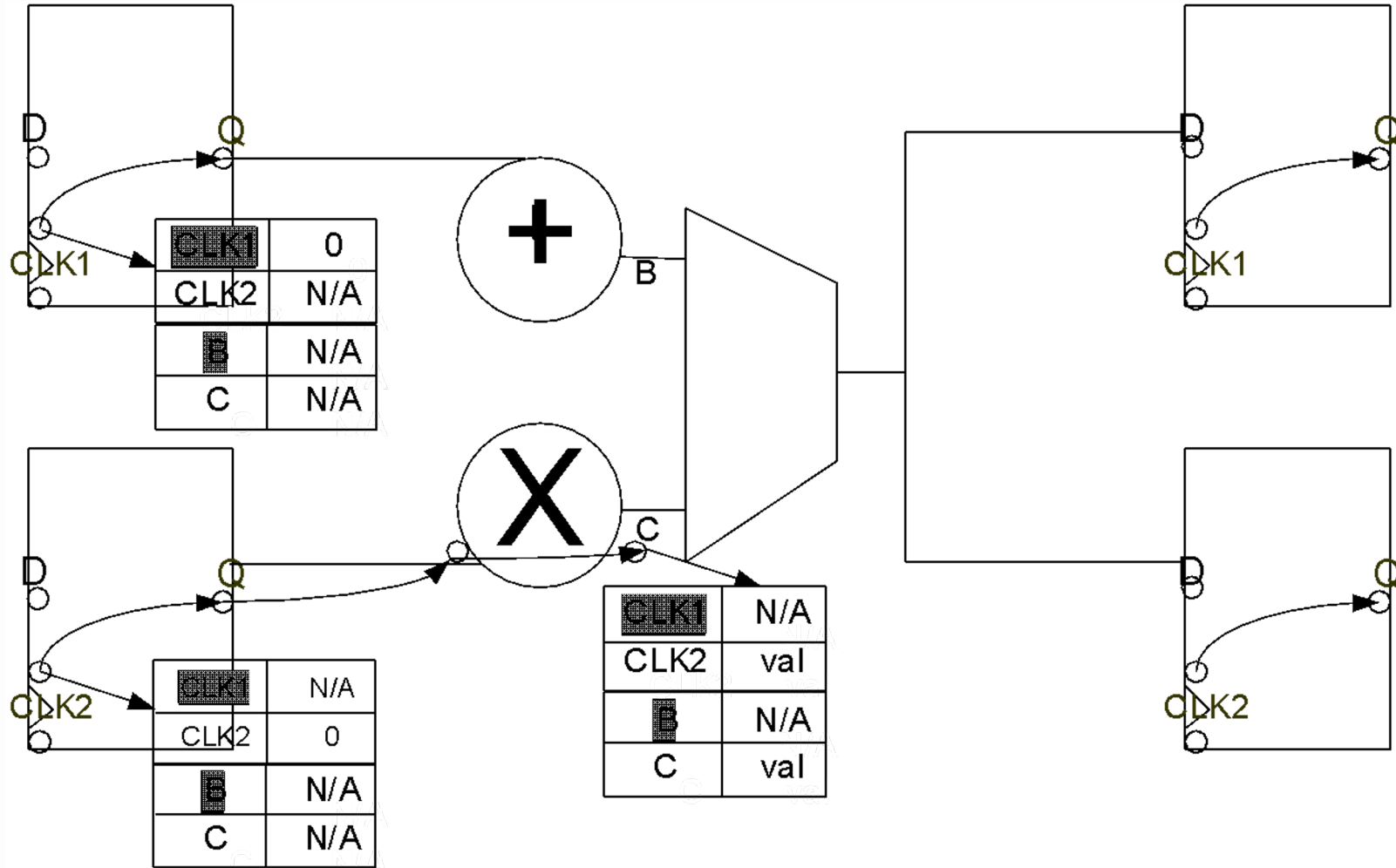
Multiple Timing Totals



Multiple Timing Totals with path completion data

- A BFS has no information about paths
- However timing exceptions are specified in terms of from, through, and to paths with a boolean expression of pins
- Mcp –from a –through {b c} –to d
- From a through b or c and also through d
- Each total can have a small state machine about what exception points it has seen
- At timing endpoints like totals with like exception point data can be combined

Through exceptions



Delay Calculation with Multiple Timing Totals

- Worst case slew merging is pessimistic but allows Delay Calculation to be a pre-process step
- If Delay Calculation is done in the BFS the critical slew merging can be done
- It is also possible for each timing total to carry its own slew to improve accuracy

Exception compression

- These two
 - M_{cp} –from a –through b –to d
 - M_{cp} –from a –through c –to d
- can be combined to
 - –from a –though $\{b\ c\}$ –to d
- This is a boolean problem to get everything into a sum of products form

Incremental Timing

- Netlist edits, full retime
- Netlist edits, fanout cone retime
- Netlist edits, query based retime
- The choice of how incremental to go depends on the optimization approach
- More global cost functions require less incrementalness
- More locally greedy approaches require more